Instructor Notes — Promises & Async/Await

How to explain this to students

The Core Mental Model: A Receipt

The best analogy for a Promise is a **coffee shop receipt**.

When you order at a busy counter, they hand you a receipt with a number. You don't wait at the counter — you go sit down. That receipt is a **Promise**: it is not the coffee, but a *guarantee that the coffee is coming* (or a notification that they ran out).

Later, one of two things happens:

- The barista calls your number → the Promise **fulfilled** (you get your coffee)
- They come tell you the machine broke → the Promise **rejected** (something went wrong)

While you were waiting, you weren't blocked — you could check your phone, talk to a friend, etc. That's the whole point: **async code doesn't block the browser while waiting**.

What is a Promise, technically?

A `Promise` is a JavaScript object that represents an operation that hasn't completed yet. It wraps an async operation and gives you a way to attach callbacks *after* the operation finishes — without needing to pass those callbacks in at the moment you start the operation.

```
new Promise((resolve, reject) => {
    // This executor function runs immediately and synchronously.
    // Start your async work here (setTimeout, fetch, file read, etc.)
    // When it finishes: call resolve(value) or reject(error)
})
```

The executor runs **right now**. The Promise object is returned **right now**. But the resolution/rejection happens *later* — that's what makes it async.

The Three States

A Promise is always in exactly one of these states:

```
┌─────────┐
│ Pending │  ── resolve(value) ──►  Fulfilled ✓
│         │  ── reject(error)  ──►  Rejected  ✗
└─────────┘
```

Pending — the async work is in progress. The Promise object exists but has no value yet.

Fulfilled — the work completed successfully. The Promise holds a value. This is also called *resolved*, though technically "resolved" can mean something slightly different in spec language. For teaching purposes: fulfilled = resolved = success.

Rejected — the work failed. The Promise holds an error/reason.

The critical rule: once a Promise is settled (fulfilled or rejected), it **cannot change state ever again**. Calling `resolve()` a second time does nothing. This is what makes Promises reliable — you will always get exactly one outcome.

.then(), .catch(), .finally()

These are how you *react* to a Promise settling:

```
somePromise
    .then(value  => { /* runs if fulfilled */ })
    .catch(error => { /* runs if rejected  */ })
    .finally(()  => { /* ALWAYS runs       */ });
```

Important: these don't run right now. They register callbacks that will run *in the future* when the Promise settles. Execution continues past this code immediately.

.then() always returns a new Promise

This is the most important thing to understand about Promises and the source of most student confusion:

```
const p2 = p1.then(value => value * 2);
```

`p2` is a **new Promise** whose resolved value is whatever you return from the callback. This is what enables chaining:

```
fetch('/api/user')          // returns Promise<Response>
    .then(res => res.json())  // returns Promise<data>  (res.json() is also async!)
    .then(data => render(data))
    .catch(err => showError(err));
```

Each `.then()` is a link in a chain. Return a plain value → next link gets it. Return a Promise → the chain **waits** for that Promise before continuing. Throw an error → execution **jumps** to the nearest `.catch()`.

The "Pyramid of Doom" Problem (Why Promises Were Invented)

Before Promises, async code was written with nested callbacks:

```
// "Callback hell" — hard to read, hard to handle errors
getUser(id, function(err, user) {
    if (err) return handleError(err);

    getPosts(user.id, function(err, posts) {
        if (err) return handleError(err);

        getComments(posts[0].id, function(err, comments) {
            if (err) return handleError(err);

            render(user, posts, comments);
        });
```

```
    });
});
```

Promises flatten this into a readable chain:

```
getUser(id)
    .then(user  => getPosts(user.id))
    .then(posts => getComments(posts[0].id))
    .then(comments => render(comments))
    .catch(handleError);   // ONE place handles all errors
```

Promise.all() — The Most Important Combinator

```
Promise.all([p1, p2, p3])
```

Fires all three at the same time (parallel, not sequential). Resolves with `[value1, value2, value3]` once *all* have fulfilled. **Rejects immediately** if *any* one rejects (fail-fast).

Why does this matter?

Compare sequential vs parallel:

```
// SEQUENTIAL — each waits for the previous (slow!)
const user    = await fetch('/user/1').then(r => r.json());    // 300ms
const posts   = await fetch('/posts').then(r => r.json());     // 300ms
const todos   = await fetch('/todos').then(r => r.json());     // 300ms
// Total: ~900ms

// PARALLEL — all fire at once (fast!)
const [user, posts, todos] = await Promise.all([
    fetch('/user/1').then(r => r.json()),
    fetch('/posts').then(r => r.json()),
    fetch('/todos').then(r => r.json()),
]);
// Total: ~300ms (limited by the slowest request)
```

Use `Promise.all()` whenever your requests are **independent** (one doesn't depend on the result of another). If they're dependent (you need the user ID to fetch their posts), they must be sequential.

Promise.all() vs Promise.allSettled()

```
// Promise.all — fail-fast, all-or-nothing
Promise.all([good, bad, good])
// → rejects immediately when 'bad' rejects, ignores the others

// Promise.allSettled — never rejects, reports every outcome
Promise.allSettled([good, bad, good])
// → resolves with:
```

```
// [
//   { status: 'fulfilled', value: ... },
//   { status: 'rejected',  reason: ... },
//   { status: 'fulfilled', value: ... }
// ]
```

Rule of thumb:

- `Promise.all` when you need **all of them** (missing any = can't render)
- `Promise.allSettled` when you want **as many as you can get** (missing some = still useful)

---

async / await: Syntactic Sugar

`async/await` is not a new feature — it is a **syntax transformation** built on top of Promises. Every `async` function returns a Promise. Every `await` is a `.then()` in disguise.

```
// Promise style
function loadUser(id) {
    return fetch(`/api/users/${id}`)
        .then(res => {
            if (!res.ok) throw new Error(`HTTP ${res.status}`);
            return res.json();
        });
}

// async/await style — identical behaviour, more readable
async function loadUser(id) {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
}
```

The JavaScript engine transforms the `async/await` version into something very close to the Promise version internally. You get the same:

- Async behaviour (doesn't block the browser)
- Same Promise states
- Same microtask queue scheduling

await pauses the function, not the browser

This is the most common misconception:

```
async function example() {
    console.log('A');
    const data = await fetch('/api/data');  // function pauses here
    console.log('C');                        // runs after data arrives
}

example();
console.log('B');   // runs IMMEDIATELY, before C
```

```
// Output: A, B, C
```

`await` pauses the *enclosing async function* and returns control to the caller. The rest of the page (other event handlers, animations, etc.) keeps running normally.

try/catch vs .catch()

With async/await, use `try/catch` instead of `.catch()`:

```
async function loadData() {
    try {
        const res  = await fetch('/api/data');
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json();
        render(data);

    } catch (err) {
        // Catches: network failure, our manual throw, JSON parse error
        showError(err.message);

    } finally {
        // Always runs — good for hiding a loading spinner
        hideSpinner();
    }
}
```

async/await with Promise.all()

They compose naturally. This is very common in React:

```
async function loadDashboard() {
    // Still fires both requests simultaneously
    const [user, posts] = await Promise.all([
        axios.get('/users/1').then(r => r.data),
        axios.get('/posts?userId=1').then(r => r.data),
    ]);

    renderDashboard(user, posts);
}
```

When to use which style?

Situation	Recommendation
Single async operation	`async/await` — cleaner
Sequential operations (each depends on the last)	`async/await` — reads like sync code
Parallel operations	`await Promise.all([...])`

"Best effort" (some may fail)	`await Promise.allSettled([...])`
Working with a library that returns raw Promises	`.then()` chaining is fine
Timeout / race condition	`Promise.race()`

Why students still need to understand raw Promises

Even though `async/await` covers 90% of day-to-day work, students will encounter raw Promises in the wild:

1. **Library code** — many APIs return Promises directly and expect you to chain them
2. **`Promise.all` / `allSettled` / `race`** — no `await`-only equivalent for these
3. **Error debugging** — "UnhandledPromiseRejection" errors only make sense if you understand Promise chains
4. **React patterns** — `useEffect` cleanup, `AbortController`, and many hooks deal with Promises directly
5. **Interview questions** — Promise internals are a classic frontend interview topic

---

Common Student Mistakes

1. Forgetting `await`

```
// BUG: data is a Promise, not the actual value
const data = fetch('/api/data').then(r => r.json());
console.log(data.title);   // undefined — data is a Promise object

// FIX
const data = await fetch('/api/data').then(r => r.json());
```

2. Not checking `res.ok` with fetch()

```
// BUG: this does NOT throw on a 404 — fetch only rejects on network failure
const res = await fetch('/api/nonexistent');
const data = await res.json();   // silently gets the 404 error body

// FIX
if (!res.ok) throw new Error(`HTTP ${res.status}`);
```

This is the main reason to prefer Axios — it throws automatically on 4xx/5xx.

3. Sequential awaits when parallel is possible

```
// SLOW — waits 300ms, then 300ms, then 300ms = 900ms total
const a = await fetchA();
const b = await fetchB();
const c = await fetchC();

// FAST — all fire simultaneously = ~300ms total
const [a, b, c] = await Promise.all([fetchA(), fetchB(), fetchC()]);
```

4. Swallowing errors silently

```
// BUG: empty catch — errors disappear with no trace
try {
    const data = await loadData();
} catch (err) {}   // never do this

// FIX: always at minimum log the error
} catch (err) {
    console.error('loadData failed:', err);
    showErrorToUser(err.message);
}
```

5. Using `async/await` inside `.forEach()`

```
// BUG: forEach does not wait for async callbacks
items.forEach(async item => {
    await processItem(item);   // these all fire but forEach doesn't wait for them
});
console.log('done');   // prints BEFORE items are processed

// FIX: use for...of, or Promise.all + map
for (const item of items) {
    await processItem(item);           // sequential
}

// OR — parallel:
await Promise.all(items.map(item => processItem(item)));
```

The Timeline: How JavaScript Executes Async Code

Students sometimes wonder why `await` doesn't just pause everything. Here's why it can't:

JavaScript is **single-threaded** — there is only one thread of execution. If we blocked that thread waiting for a network response, the entire page would freeze: no scrolling, no button clicks, no animations.

Instead, JavaScript uses an **event loop**:

1. Run the current synchronous code until it's done
2. Check the queue: is any async operation ready? (timer fired, response arrived)
3. Run that callback
4. Repeat

`await` works by:

1. Starting the async operation (e.g. `fetch()`)
2. **Suspending the current function** and returning control to the caller
3. Adding a callback to run "when the operation is done"
4. When the response arrives, the event loop resumes the function from where it left off

The browser stays responsive the whole time because the thread was never blocked.

Quick Reference

```
// Create
new Promise((resolve, reject) => { ... })
Promise.resolve(value)    // already-fulfilled Promise
Promise.reject(error)     // already-rejected Promise

// Consume
.then(value  => ...)
.catch(error => ...)
.finally(()  => ...)

// Combinators
Promise.all([p1, p2, p3])          // wait for all; fail-fast
Promise.allSettled([p1, p2, p3])   // wait for all; report each outcome
Promise.race([p1, p2, p3])         // first to settle wins

// async/await
async function fn() {
    try {
        const value = await somePromise;
    } catch (err) { ... }
    finally { ... }
}
```